

IntentSpec: How Much Does LLM-generated Code Violate Developer Intent?

Anonymous ACL submission

Abstract

Code generated by LLMs can violate a developer’s implicit intentions when given an ambiguous prompt, yet standard benchmarks measure only whether code passes its stated tests. We introduce the Intent Violation Rate (IVR) and a 49-problem pilot benchmark derived from HumanEval+. Each problem strips implicit constraints from a clarified prompt and encodes them as hidden constraint tests. IVR measures the fraction of LLM-generated solutions that pass the stated (visible) tests yet fail hidden constraint tests that capture unstated intent. Evaluating Claude Sonnet 4.6, we find it passes 94.3% of stated tests but violates intent in 54.5% of problems, even when the code passes every stated test. These violations follow a systematic, bimodal pattern. Our findings indicate that pass rates overstate how well generated code reflects developer intent.

1 Introduction

When LLMs are tasked with building code, they can understand the main task at hand, such as writing a function or script. However, these outputs are not always fully correct, requiring human-in-the-loop review and adjustments before they can be deployed. Developers, through review, will capture functional or logical errors, and cases where the code does something different from what they intended. Current benchmarks measure functional correctness of LLM generated code via a test pass rate (Austin et al., 2021), however, this metric does not capture whether the solution fully reflects what the developer intended.

Test pass rates alone are insufficient as a metric because they do not detect intent violations, which refer to scenarios where LLMs will provide outputs that satisfy stated tests, but violate implicit constraints the developer assumes will be accounted for, and did not explicitly state. Prior work has documented similar failures in which LLMs produce code that will game specifications and exploit

evaluation loopholes to pass tests without solving the intended task (Zhong et al., 2025). We focus on a distinct failure mode: outputs that legitimately pass valid stated tests, but still violate unstated developer intent. To measure this, we propose the Intent Violation Rate (IVR), a metric that quantifies whether an output violates developer intent or not. We evaluate IVR on 49 algorithm specification problems, and demonstrate that Claude Sonnet 4.6 violates implicit developer constraints in approximately 54.5% of cases, even when solutions pass stated tests.

2 Related Work

When it comes to LLMs generating code, a big concern is when developers take the outputted code at face value due to a high pass rate on tests they built. Prior work defines code that passes provided test cases as having functional correctness (Austin et al., 2021), however Chen et al. (2021) also note that models can pass weak test suites simply by deleting failing functionality. It has also been found that LLM generated code would often "overfit to assert statements", rather than properly apply logic (Austin et al., 2021). This meant LLMs would hard-code outputs for visible tests rather than actually create an accurate function that properly handled the general task at hand. A growing body of work also details LLMs that exploit evaluation shortcuts in code generation; this is seen in test suites where tasks were constructed to purposefully contradict specifications (Zhong et al., 2025), or where outputs are hard coded, or tests were edited to pass (Gabor et al., 2025). Despite the focus on deliberate exploitation of flawed or gameable test suites, they further show that solutions passing held-out tests can still be incorrect. We build on this observation to measure violations of unstated intent in generated code that passes valid stated tests.

By default, models rarely ask for clarification

when a prompt is unclear, so when it comes to giving LLMs ambiguous prompts, prior work has seen a significant drop in test pass rates by approximately 80% (Li et al., 2026). It shows that functional correctness (Pass@1) will fall apart when ambiguity is introduced to prompts, and we measure the intent violated in code where functional correctness holds. However, to address the drop in test pass rates, other work finds that by asking clarifying questions, the test pass rate of GPT-4’s generated code saw an increase of approximately 10 points, rising from 70.96% to 80.80% in its functional correctness (Mu et al., 2023). Additionally, Kim et al. (2024) finds that training models to recognize ambiguity and to ask, rather than guess, shows improvements in ambiguity detection. Rather than utilize methods to resolve ambiguity through interaction or prompt repair, IVR measures the intent violations that remain in code that passes its stated tests, generated from ambiguous prompts.

3 Measuring Intent Violation Rate

3.1 Definition

Intent Violation Rate, or IVR, is defined as follows:

$$\text{IVR} = \frac{n_{\text{violating}}}{n_{\text{stated passed}}} \quad (1)$$

Where for a given solution:

$n_{\text{stated passed}}$: number of generated solutions passing all stated tests

$n_{\text{violating}}$: subset of those that fail ≥ 1 hidden constraint test

IVR can be interpreted as: "Of the solutions that satisfy the given spec’s visible tests, what fraction violates the developer’s actual intent?" It is only measured over solutions that pass visible stated tests, therefore any solutions that do not pass the initial public test are not included in the calculation. The reported IVR is the mean of per-problem IVRs over problems that had ≥ 1 solution passing stated test C1.

3.2 Benchmark Construction

The built benchmark consists of 49 algorithm problems derived from HumanEval+ (Liu et al., 2023). Each problem consists of an ambiguous prompt, a gold prompt, a stated test (C1), and hidden constraint tests (C2–C4, the number varying per problem).

The selected HumanEval+ problems all contained a gold prompt, which we stripped down to produce an ambiguous prompt. In some cases, additional specifics were added to the gold prompt to further clarify the task at hand. This gold prompt was always hidden to the LLM, as it only received the ambiguous prompt, and C1, the stated test. Each aspect that was stripped from the gold prompt to formulate the ambiguous prompt was then used to create the hidden constraint tests.

The scope was narrowed down to algorithm problems, where the prompt described a self contained function. A subset of these problems were cleanly decomposable into executable tests, and the formulation of the ambiguous prompt came from under specifying rather than misleading. Other task types, such as data transformation, involve fuzzier constraints and lack a comparable source at scale, being left to future work.

As an example, the original HumanEval/34 prompt reads: "Return sorted unique elements in a list."

We decompose it as shown in Table 1:

HumanEval/34	
Ambig.	Return the unique elements in a list.
Gold	Return only the unique values, sorted in ascending order.
C1	<i>stated</i> : only unique values solution([-2,4,4,6,6]) == [-2,4,6]
C2	<i>hidden</i> : sorted ascending solution([-2,0,5,3,5,0,3]) == [-2,0,3,5]

Table 1: Example spec pair. Stripping the sort-order requirement from the gold prompt yields the ambiguous prompt; C1 tests what remains stated, C2 tests the stripped intent.

To ensure the benchmark is sound, we applied two validation checks. First, a leakage audit was performed. When building the stated test, it must be satisfiable from the ambiguous prompt alone. During initial construction, 11 of the 49 problems failed this check. 9 of the failed tests were due to type errors, output format contamination, and hidden constraints leaking into the stated test. Of the hidden constraint leakage, what occurred was that the stated test also check a hidden constraint test, so a solution could not pass without already knowing part of the developer’s intent. The other 2 problems were identified in the second validation check, where the canonical solution was run

against the entire test suite to ensure stated and hidden tests were sound. These issues were errors in the tests themselves, something the initial leakage audit could not catch. Across all 49 problems, the validated references yield an IVR = 0, confirming that the hidden tests are satisfiable by correct code, and are not themselves the source of the measured violations.

3.3 Evaluation Pipeline

The pipeline occurs in 4 stages: generation, execution, pass/fail determination, and IVR calculation.

Generation: Using Claude Sonnet 4.6, solutions are generated from the ambiguous prompt and the stated test C1 at the API default temperature of 1.0. All other information (e.g, gold clarified prompt and hidden tests) is not shown to the LLM.

Execution: For each problem, 5 solutions are produced, with a 5-second timeout per test, returning a pass/fail result. This is adapted from HumanEval’s own execution pipeline, which runs candidate solutions in process isolation with syscall lockdown and timeouts (Chen et al., 2021).

Pass/Fail Determination: Each solution for each problem is run against the stated test C1 and hidden tests (C2–C4), producing a per-solution result for each problem. IVR is then calculated for that specific problem.

Overall IVR Calculation: Each problem produces its own IVR value, and the overall IVR value is the mean across N problems with at least one stated test passing solution.

4 Experiments

4.1 Setup

We evaluated Claude Sonnet 4.6 following the pipeline in Section 3.3, generating 5 solutions per problem at temperature 1.0 with a 1024 token limit. This is a single-model evaluation; we leave additional models to future work.

4.2 Results

Across the stated tests, Claude Sonnet 4.6 passes 94.3% of the time, but violates at least one hidden intent constraint in 54.5% of problems. This highlights the high functional correctness of LLM-generated code that nonetheless violates unstated intent. Of the 49 benchmark problems, two (HE/101, HE/112) had no solution passing the stated test C1 and are therefore excluded from the

IVR calculation, which is computed over the remaining 47.

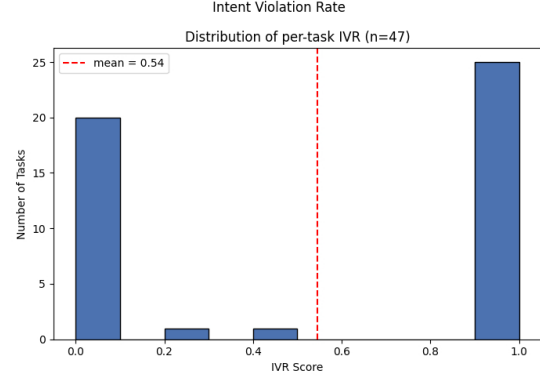


Figure 1: Distribution of per-problem IVR for Claude Sonnet 4.6 across 47 spec pairs. Two problems with no stated-test-passing solutions are excluded.

Figure 1 shows a breakdown of the problems that pass the stated test and their level of violation. 45 of the 47 problems that pass C1 are at the extremes.

The mean IVR of 54.5% reflects the bimodal split we see, with problems tending to cluster at IVR = 0 (no solutions violate intent) or IVR = 1 (every solution violates intent), with 45 out of 47 at these extremes. This near-deterministic behavior indicates that intent violations are systematic properties of a given task rather than sampling noise. If violations came from random variation in generation, per-problem IVR would be spread across intermediate values. Instead, the model consistently produces outputs that either fail to capture hidden constraints or fully do so.

The stated test C1, fails significantly less than hidden tests, at 5.7%. Compared to hidden test fail rates, C1 is roughly 37 points lower than C2 (42.9%), and is about 10 points lower the least-violated hidden constraint, C4 (15.4%).

Constraint	Present	Failed	Fail Rate
C1 (stated)	245	14	5.7%
C2 (hidden)	231	99	42.9%
C3 (hidden)	201	58	28.9%
C4 (hidden)	65	10	15.4%

Table 2: Failure rate by constraint for Claude Sonnet 4.6 across 49 algorithm spec pairs (245 solutions). The stated test (C1) is rarely failed, but hidden intent constraints fail far more often. “Present” counts solutions whose problem included the constraint; C2–C4 rates are over stated-test-passing solutions.

4.3 Analysis

Hidden constraints are numbered in the order they were decomposed from the gold prompt, and this ordering is positional and does not strictly rank constraints by centrality. We nonetheless observe that earlier numbered constraints fail more often than later ones (Table 2), which may indicate that the first stripped constraint is often most central to the task, while later constraints often capture narrower edge cases.

If we refer to HumanEval 34 in Section 3.2, it has a calculated IVR of 1.0, with all 5 solutions passing the stated test C1: return unique values, but those same solutions all failed the hidden constraint C2: sorted ascending order. However, in HumanEval 25, the ambiguous prompt reads as "Write a function named solution that returns the prime factors of a given integer", and this problem also contains the hidden constraint of returning the output in ascending order, and across all 5 solutions, it passes this constraint and others, reporting a IVR of 0.0. This suggests that intent satisfaction is constraint-dependent and context-dependent, rather than uniform.

5 Conclusion

We introduced the Intent Violation Rate (IVR), a metric that measures how often LLM generated code passes its stated tests, while violating unstated developer intent, and a small pilot benchmark of N=49 audited algorithm specification pairs derived from HumanEval+. Evaluating Claude Sonnet 4.6 on this benchmark, we found that despite 94.3% of solutions passing stated tests, the model produced code that violated at least one hidden intent constraint in 54.5% of problems. These violations fell into a systematic and near-deterministic pattern, rather than random noise, indicating that functional correctness metrics can overstate how well generated code reflects what a developer actually intended. Future work includes detecting underspecification before generation, expanding the benchmark, and extending IVR across additional models and task domains.

Limitations

Our experiment and benchmarking was only tested on the Claude Sonnet 4.6 model, at a single temperature of 1.0. IVR could differ across models and lower temperatures. Additionally, the benchmark is small, with only N=49, limiting statisti-

cal power, with constraint type distributions being more suggestive, than robust. The benchmark also only focuses on Python algorithmic problems only, other languages and other problem types such as data transformation may have different findings. The problem specification is constrained to single-annotation construction, therefore the defined behavior of clarified gold prompts and tests reflect the judgment of one person. This also affects the operationalized definition of "intent", as we do not measure "intent" in an absolute metric. Also, HumanEval+ is a public dataset, it is possible that the model has seen the original problems in training prior to our experimentation.

Ethical Considerations

Our work highlights a failure mode with direct deployment implications: LLM-generated code can pass its stated tests while silently violating unstated developer intent. As LLM-assisted coding is increasingly integrated into software workflows, a high test pass rate may give developers false confidence in code that does not do what they intended. We frame IVR as a measurement that makes this gap measurable, and we caution against treating functional-correctness metrics as sufficient evidence of correctness in deployment.

Our benchmark is derived from HumanEval+ (Liu et al., 2023), which is publicly available, so it is possible the evaluated model encountered the original problems during training. We discuss this contamination risk in the Limitations section. We introduce no new human-subjects data, and the benchmark contains only algorithmic programming problems with no personal, sensitive, or identifying information.

Finally, our operational definition of intent reflects the judgment of a single annotator constructing the ambiguous and gold prompts and the test suites. Any measure of intent is inherently normative, and our benchmark encodes one reasonable interpretation of each problem rather than a ground truth. We therefore present IVR as a relative diagnostic, not an absolute measure of whether LLM-generated code is "correct".

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and

Charles Sutton. 2021. [Program synthesis with large language models](#). *arXiv preprint arXiv:2108.07732*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. [Evaluating large language models trained on code](#). *arXiv preprint arXiv:2107.03374*.

Jonathan Gabor, Jayson Lynch, and Jonathan Rosenfeld. 2025. [EvilGenie: A reward hacking benchmark](#). *arXiv preprint arXiv:2511.21654*.

Hyuhng Joon Kim, Youna Kim, Cheonbok Park, Junyeob Kim, Choonghyun Park, Kang Min Yoo, Sang-goo Lee, and Taeuk Kim. 2024. [Aligning language models to explicitly handle ambiguity](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1989–2007, Miami, Florida, USA. Association for Computational Linguistics.

Jialin Li, Yuan Wu, and Yi Chang. 2026. [ClarEval: A benchmark for evaluating clarification skills of code agents under ambiguous instructions](#). *arXiv preprint arXiv:2603.00187*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. [Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572.

Fangwen Mu, Lin Shi, Song Wang, Zhuohao Yu, Binqun Zhang, Chenxue Wang, Shichao Liu, and Qing Wang. 2023. [ClarifyGPT: Empowering LLM-based code generation with intention clarification](#). *arXiv preprint arXiv:2310.10996*.

Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. 2025. [ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases](#). *arXiv preprint arXiv:2510.20270*.

A Spec Pair Construction Details

Construction Protocol: - Start from a HumanEval+ problem and its original prompt - Identify the implicit constraints the original problem specifies (ordering, edge-case handling, conditional behavior, etc.) - Write a gold (clarified) prompt built off the original prompt that states all constraints explicitly; in some cases add specifics beyond the original to remove residual ambiguity - Strip the constraints from the gold prompt to produce the ambiguous prompt; omitting information that does not steer toward a wrong interpretation - Encode each stripped constraint as a hidden test (C2–C4); C1 is a stated test that must be satisfiable from the ambiguous prompt alone - Problems are algorithm only as they

cleanly decompose into executable tests; other task types involve fuzzier constraints and lack a comparably licensed source at scale

HumanEval/88 (conditional sort, IVR = 1.0)

Ambig.	Return a sorted copy of a list of non-negative integers, without modifying the original.
Gold	Sort ascending if the sum of the first and last elements is odd, descending if even; do not mutate the input.
C1	<i>stated</i> : sorts (odd sum) solution([2,4,3,0,1,5]) == [0,1,2,3,4,5]
C2	<i>hidden</i> : descending on even sum solution([2,4,3,0,1,5,6]) == [6,5,4,3,2,1,0]

Table 3: HE/88: the model sorts but ignores the parity-dependent direction. All 5 solutions violate C2.

HumanEval/161 (fallback, IVR = 1.0)

Ambig.	Swap the case of each letter, leaving non-letter characters unchanged.
Gold	Swap the case of each letter; if the string contains no letters, return it reversed instead.
C1	<i>stated</i> : case swap solution('a@c') == 'A@c'
C2	<i>hidden</i> : reverse if no letters solution('1234') == '4321'

Table 4: HE/161: the model performs the case swap but misses the no-letters fallback. All 5 solutions violate C2.

Validation Detail

The benchmark was validated in two passes.

Leakage audit. Each stated test (C1) must be satisfiable from the ambiguous prompt alone, without knowledge of any hidden constraint. Of the 49 problems, 9 initially failed this check: type errors, output-format contamination (e.g., a list expected where a tuple was returned), and hidden-constraint leakage, where C1 inadvertently tested a hidden constraint so that no solution could pass C1 without already encoding part of the intent. All 9 were corrected.

Canonical soundness check. We ran the HumanEval+ canonical solution for each problem against the full test suite (C1 and all hidden constraints); a correct solution must pass all constraints, yielding IVR = 0. This surfaced 2 further problems whose tests were themselves faulty rather

HumanEval/25 (satisfied, IVR = 0.0)	
Ambig.	Return the prime factors of a given integer.
Gold	Return the prime factors in ascending order, with repeated factors and product equal to the input.
C1	<i>stated</i> : basic factorization solution(13) == [13]
C2	<i>hidden</i> : ascending order solution(15) == [3,5]

Table 5: HE/25: the ascending-order constraint is satisfied by all 5 solutions, as standard factorization emits factors in ascending order. Contrast with HE/34, where the analogous ordering constraint is violated.

than leaked: HE/47 had an incorrect expected median value, and HE/18 contained a contradictory empty-substring constraint. Both were corrected (the HE/18 constraint was removed, as the empty-substring convention is ambiguous). After correction, all 49 canonical references yield IVR = 0, confirming the hidden tests are satisfiable by correct code.

B Prompts and Sandbox Configuration

Write a Python function named ‘solution’ that solves the following task. prompt Requirements: - Name the function exactly ‘solution’. - Return only the function definition — no explanation, no markdown, no imports unless the function itself needs them.

Where prompt is the ambiguous prompt.

Decoding Parameters: - Model: Claude Sonnet 4.6 - Temperature: 1.0 (API default) - Max Tokens: 1024 - Number Samples per Problem: 5

Post Processing: Markdown code fences are stripped from model output before execution to prevent ...

Sandbox: Subprocess-based execution, one subprocess per test, 5-second timeout per test, adapted from HumanEval’s execution harness (process isolation, syscall restrictions). Each solution is run against C1 (stated test) and each hidden constraint independently.

C Per-Problem Results

Table 6 reports the per-problem IVR for all 49 spec pairs, the number of solutions passing the stated test (C1) out of 5, and which hidden constraints were violated. HE/101 and HE/112 had no C1-passing solutions and are excluded from the IVR mean.

Problem	C1 pass	IVR	Violated
HE/7	5/5	0.00	–
HE/12	5/5	0.40	C3
HE/18	5/5	0.00	–
HE/20	5/5	0.00	–
HE/21	5/5	0.00	–
HE/25	5/5	0.00	–
HE/26	5/5	0.00	–
HE/30	5/5	0.00	–
HE/34	5/5	1.00	C2
HE/42	5/5	0.00	–
HE/47	5/5	0.00	–
HE/51	5/5	0.00	–
HE/57	5/5	1.00	C2
HE/58	5/5	1.00	C2,C3
HE/64	5/5	1.00	C2,C3
HE/65	5/5	1.00	C3
HE/66	5/5	1.00	C2
HE/68	5/5	0.00	–
HE/70	5/5	0.00	–
HE/71	5/5	1.00	C2,C3
HE/72	5/5	1.00	C2
HE/74	5/5	0.00	–
HE/86	5/5	1.00	C2,C3
HE/88	5/5	1.00	C2
HE/90	5/5	1.00	C3,C4
HE/91	5/5	1.00	C2,C3
HE/93	5/5	1.00	C2,C3
HE/95	5/5	1.00	C4
HE/98	5/5	0.00	–
HE/99	5/5	0.00	–
HE/101	0/5	–	–
HE/102	5/5	1.00	C3
HE/104	5/5	1.00	C2
HE/105	5/5	0.20	C2
HE/108	1/5	0.00	–
HE/111	5/5	1.00	C2,C3
HE/112	0/5	–	–
HE/120	5/5	0.00	–
HE/126	5/5	1.00	C3
HE/131	5/5	0.00	–
HE/135	5/5	0.00	–
HE/136	5/5	0.00	–
HE/140	5/5	1.00	C2
HE/145	5/5	1.00	C2
HE/149	5/5	1.00	C2
HE/151	5/5	1.00	C2
HE/158	5/5	1.00	C2
HE/161	5/5	1.00	C2
HE/163	5/5	1.00	C2,C3

Table 6: Per-problem IVR across all 49 spec pairs. “C1 pass” is the number of the 5 sampled solutions passing the stated test; “IVR” is the fraction of those that violate ≥ 1 hidden constraint; “Violated” lists the hidden constraints failed by any C1-passing solution.